COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010 / Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	October 20, 2026	**DATE SUBMITTED:**	October 22, 2026
NAME:	Torres, Carl Andrei D.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. HOA 14.1 Parallel Algorithms and Multithreading

1. PROCEDURE OUTPUT

What procedures did you perform?

Algorithm	Best Case	Worst Case	Analysis
Algorithm 1	$O(n^2)$ (Standard Implementation) or $O(n)$ (For Pre-Sorted Arrays)	$O(n^2)$	In this algorithm, the **Bubble Sort has a best case time complexity of $O(n^2)$ for the standard implementation or $O(n)$ for pre–sorted array inputs**. Also, the **worst case time complexity is $O(n^2)$ which is for dealing with unsorted or reverse-sorted inputs**.
Algorithm 2	$O(n)$	$O(n^2)$	In this algorithm, the **Insertion Sort has a best case time complexity of $O(n)$ for the standard implementation**. Also, the **worst case time complexity is $O(n^2)$ which is the same for bubble sort (Algorithm 1) for dealing with unsorted or reverse-sorted inputs**.

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

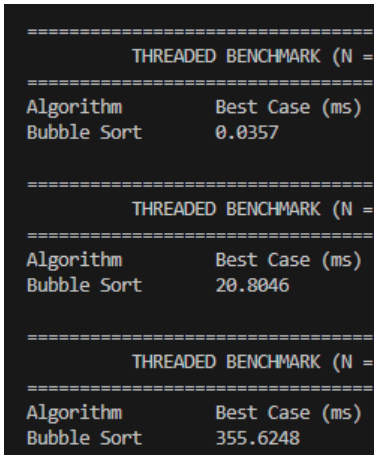
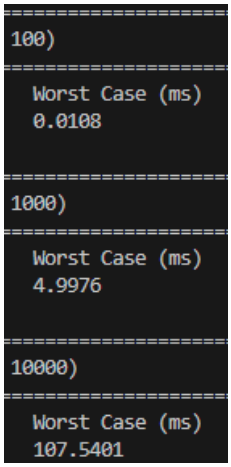
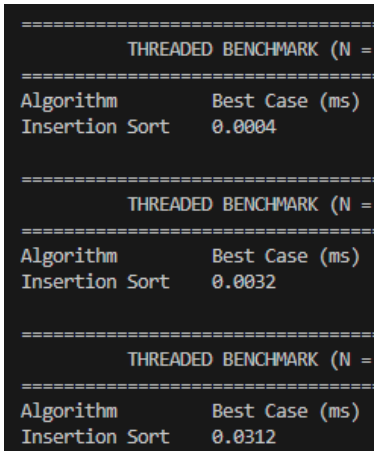
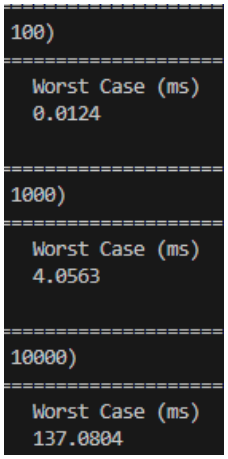
Algorithm	Best Case	Worst Case
Algorithm 1		
Algorithm 2		

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

Analysis	In this procedure, I've **analyzed that the theoretical and experimental performance between the Bubble Sort (Algorithm 1) and Insertion Sort (Algorithm 2) under the different input sizes but having a static and fixed dataset using the static dataset generator and multithreaded benchmarking technique**. Theoretically, **both of the sorting algorithms have the worst-case time complexity of O(n^2) because of the unsorted and reverse-sorted inputs and best time complexity of O(n) if its for pre-sorted arrays**. In the Experimental Analysis, **Insertion Sort had the superior best-case efficiency than Bubble Sort, which takes about 0.0004 ms (N = 100), 0.0032 ms (N = 1,000), and 0.0312 ms (N =**

	10,000) to sort the inputs as compared to Bubble Sort which takes about 0.0357 ms, 20.8046 ms, and 355.6248 ms which it requires to pass overhead in comparison loops for swapping. For worst-case running time, the **execution time of both algorithms scaled with N when the input size has been increased tenfold as Bubble Sort increased from 4.9976 ms to 107.5401 ms and Insertion Sort from 4.0563 ms to 137.0804 ms, which is related to the theoretical worse-case time complexity of O(n^2)**.

2. Supplementary Activity

ILO A: Contrast and compare the two searching techniques used in class. The required deliverable for this activity would be the theoretical and empirical analysis.
For the theoretical analysis, the deliverable must be:

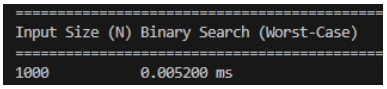
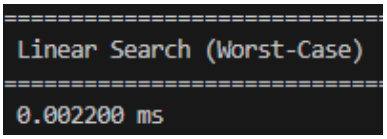
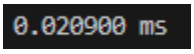
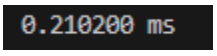
- Pseudocode Analysis
- Time and Space Complexities

For the empirical analysis, the deliverable must be the filled-out table and your analysis

Pseudocode	**Pseudocode Analysis**
Algorithm LinearSearch(array, target): For each index i from 0 to length(array) - 1: If array[i] == target: Return i // Target found at index i Print "The element is not found" Return -1 // Target not found	The Linear Search algorithm **works by sequentially going through the array with a For loop starting from index i = 0 to i = length(array) - 1**. Inside the loop, an **If condition is made to check if the element at index i (array[i]) is equal to the specified number**. **Once a match is found, the loop is exited and the function returns Return i to indicate the position of the target**. Otherwise, **if all elements in the array have been scanned and no matching element is found, the program goes to print "The element is not found" and returns Return -1 to show that the target was not in the array**.
Algorithm BinarySearch(array, target): Set left = 0 Set right = length(array) - 1 While left <= right: Set mid = left + (right - left) / 2 // Find middle index If array[mid] == target: Return mid // Target found at index mid Else If array[mid] < target:	The Binary Search algorithm **executes a divide-and-conquer search on a pre-sorted array by initializing pointer variables left = 0 and right = length(array) - 1 to define the search interval. It enters a While loop that continues as long as left <= right, calculating the middle index mid = left + (right - left) / 2 during each pass**. The **algorithm then evaluates array[mid] which if it matches the target, it immediately returns mid**. If **array[mid] is less than the target, it shifts the lower bound (left = mid + 1) to search the right half; otherwise, it updates the upper bound (right = mid - 1) to focus**

Set left = mid + 1 // Target is in the right half Else: Set right = mid - 1 // Target is in the left half Print "The element is not found" Return -1 // Target not found	**on the left half.** If the **search window collapses without locating the target, the loop terminates, prints "The element is not found", and executes Return -1**. .

Algorithm	**Time Complexity**	**Space Complexity**
Binary Search Arr	Best Case : O(1) Worst Case : O(log n)	O(1)
Linear Search Arr	Best Case : O(1) Worst Case : O(n)	O(1)

INPUT SIZE (N)	**Elapsed Time for Binary Search (Worst-Case)**	**Elapsed Time for Linear Search (Worst-Case)**
1,000		
10,000		
100,000		

Analysis	For an **input size of N = 1,000**, the **output console shows a worst-case execution time of 0.002200 ms for Linear Search and 0.005200 ms for Binary Search**. At this **smaller dataset scale**, **Linear Search executes slightly faster due to its lower initial computational overhead compared to the pointer arithmetic and midpoint divisions that is required by Binary Search**. Consequently, **as the input size expands significantly, Linear Search scales linearly O(n), while Binary Search scales logarithmically O(log n), making Binary Search vastly superior for large-scale datasets**.

Conclusion/Learnings/Analysis	In this Hands-on Activity, **I learned about the importance of theoretical asymptotic complexity in practice by testing sorting/searching algorithms on different input sizes**. In the Procedure, the **benchmarking results confirmed that if Bubble Sort and Insertion Sort have the same worst-case quadratic complexity O(n^2), then the later part has significantly better best-case scenario and requires fewer passes**. In the Supplementary Activity, the **testing with the two searching algorithms highlighted a critical difference in performance between an Binary Search algorithm with logarithmic O(log n) and Linear Search has a linear O(n) worst-case time complexity**. To improve my performance in the next laboratory activities, I will **try to limit the number of background processes that are running on the computer during the timing tests and practice the implementation of multithreaded algorithms**.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **N/A**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.